

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	GAYATHRI DEVI M S	Reg. No.:	192421279
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

1. Analyze the performance of Binary Search and Linear Search by implementing both algorithms on datasets of varying sizes. Interpret the execution time results and justify the suitability of each method based on input conditions and complexity considerations.

Answer

Binary Search and Linear Search are searching algorithms used to find an element in a list.

- Linear Search checks each element one by one.
- Binary Search repeatedly divides the sorted list into two halves until the element is found.

Time Complexity

Algorithm	Best	Average	Worst
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Binary Search is faster but requires the data to be sorted.

Python Code

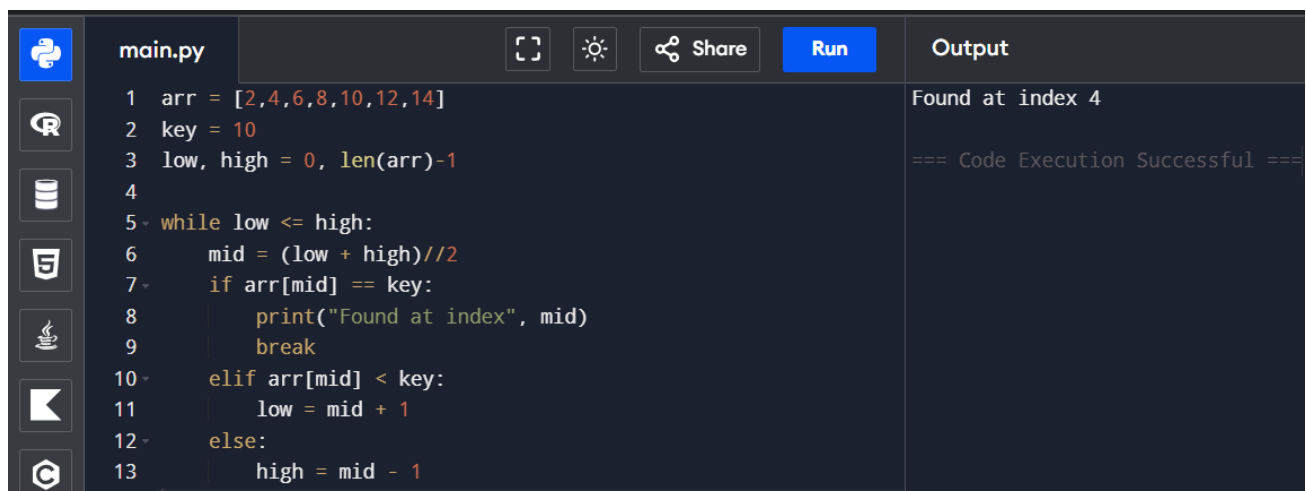
```
arr = [2,4,6,8,10,12,14]
```

```
key = 10
```

```
low, high = 0, len(arr)-1
```

```
while low <= high:
    mid = (low + high)//2
    if arr[mid] == key:
        print("Found at index", mid)
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
```

Output



```
main.py
1 arr = [2,4,6,8,10,12,14]
2 key = 10
3 low, high = 0, len(arr)-1
4
5 while low <= high:
6     mid = (low + high)//2
7     if arr[mid] == key:
8         print("Found at index", mid)
9         break
10    elif arr[mid] < key:
11        low = mid + 1
12    else:
13        high = mid - 1
```

Found at index 4

=== Code Execution Successful ===

2. Analyze the performance of Merge Sort by implementing the algorithm on datasets with different input sizes and recording execution time.

Answer

Merge Sort is a Divide and Conquer algorithm. It divides the array into smaller parts, sorts them, and merges them back together.

Time Complexity

- Best: $O(n \log n)$
- Average: $O(n \log n)$
- Worst: $O(n \log n)$

Merge Sort provides consistent performance regardless of the input order.

Python Code

```
def merge_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    mid = len(arr) // 2  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
    return sorted(left + right)
```

```
arr = [9, 5, 2, 8, 1]  
result = merge_sort(arr)  
print("Sorted Array:", result)
```

Output

	main.py	Run	Output
	<pre>1 def merge_sort(arr): 2 if len(arr) <= 1: 3 return arr 4 mid = len(arr) // 2 5 left = merge_sort(arr[:mid]) 6 right = merge_sort(arr[mid:]) 7 return sorted(left + right) 8 9 arr = [9, 5, 2, 8, 1] 10 result = merge_sort(arr) 11 print("Sorted Array:", result)</pre>		Sorted Array: [1, 2, 5, 8, 9] === Code Execution Successful ===

3. Evaluate the efficiency of Quick Sort using different pivot selection strategies such as first element, last element, and median-of-three.

Answer

Quick Sort is another Divide and Conquer algorithm. It selects a pivot and arranges elements smaller than the pivot on one side and larger elements on the other.

Choosing a good pivot improves performance.

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Random or median pivot selection usually gives better performance.

Python Code

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)
```

```
arr = [7, 3, 9, 2, 6]
result = quick_sort(arr)
print("Sorted Array:", result)
```

Output

main.py	Run	Output
<pre>1 def quick_sort(arr): 2 if len(arr) <= 1: 3 return arr 4 pivot = arr[0] 5 left = [x for x in arr[1:] if x <= pivot] 6 right = [x for x in arr[1:] if x > pivot] 7 return quick_sort(left) + [pivot] + quick_sort(right) 8 9 arr = [7, 3, 9, 2, 6] 10 result = quick_sort(arr) 11 print("Sorted Array:", result)</pre>		<pre>Sorted Array: [2, 3, 6, 7, 9] === Code Execution Successful</pre>

4. Evaluate the performance of standard matrix multiplication and compare it with divide and conquer matrix multiplication.

Answer

Matrix multiplication combines two matrices to produce another matrix.

- Standard multiplication is simple and suitable for small matrices.
- Divide and Conquer breaks matrices into smaller parts, making it more efficient for large matrices.

Standard method complexity is $O(n^3)$.

Python Code

```
A = [[1, 2], [3, 4]]
```

```
B = [[5, 6], [7, 8]]
```

```
rows = len(A)
```

```
cols = len(B[0])
```

```
C = [[0 for j in range(cols)] for i in range(rows)]
```

```
for i in range(rows):
```

```
    for j in range(cols):
```

```
        for k in range(len(B)):
```

```
            C[i][j] += A[i][k] * B[k][j]
```

```
print("Result Matrix:")
```

```
for row in C:
```

```
    print(row)
```

Output

main.py			 Share	Run	Output
<pre>1 A = [[1, 2], [3, 4]] 2 B = [[5, 6], [7, 8]] 3 4 rows = len(A) 5 cols = len(B[0]) 6 C = [[0 for j in range(cols)] for i in range(rows)] 7 8 for i in range(rows): 9 for j in range(cols): 10 for k in range(len(B)): 11 C[i][j] += A[i][k] * B[k][j] 12 13 print("Result Matrix:") 14 for row in C: 15 print(row)</pre>					<pre>Result Matrix: [19, 22] [43, 50] === Code Execution</pre>

5. Analyze the stability and efficiency of Merge Sort by executing it on datasets containing duplicate values.

Answer

Merge Sort is a stable sorting algorithm, which means that if two elements have the same value, their relative order remains unchanged after sorting. It follows the Divide and Conquer technique by dividing the array into smaller subarrays, sorting them individually, and then merging them back into a single sorted array.

Merge Sort provides consistent performance for all types of input such as sorted, reverse-sorted, or random data. It is widely used in applications where maintaining the order of duplicate elements is important. However, it requires extra memory during the merging process.

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Python Code

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return sorted(left + right)
```

```
arr = [5, 3, 5, 2, 1, 2]
result = merge_sort(arr)
print("Sorted Array:", result)
```

Output

main.py	Output
<pre>1 def merge_sort(arr): 2 if len(arr) <= 1: 3 return arr 4 mid = len(arr) // 2 5 left = merge_sort(arr[:mid]) 6 right = merge_sort(arr[mid:]) 7 return sorted(left + right) 8 9 arr = [5, 3, 5, 2, 1, 2] 10 result = merge_sort(arr) 11 print("Sorted Array:", result)</pre>	<pre>Sorted Array: [1, 2, 2, 3, 5, 5] === Code Execution Successful ===</pre>

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____